

Project proposal

Max Arlestål Hoffmann ma226pv

October 2025

1 Objective

During my bachelor thesis project on developing a self-service web application, I experienced the complexity of maintaining consistency between API documentation, implementation, and test artifacts. This inspired the current project proposal: a model driven solution for automated test case generation and synchronization for web APIs using OpenAPI specifications. This project demonstrates how MDE can automate test generation, improve traceability between models and code and ensure continuous synchronization across development artifacts.

1.1 Problem Formulation and Domains

Two different domains:

1. **API Documentation Domain:** Contains OpenAPI specifications that describe REST APIs, including endpoint definitions, request/response schemas, and authentication methods used when performing client API calls.
2. **Software Testing Domain:** Encompasses test cases and test execution procedures for validating different API calls and their expected behaviors.

The problem addressed is the challenge developers face in maintaining API specifications (such as Swagger/OpenAPI documentation files) separately from their test code. When API specifications and calls change, the corresponding test files must be manually updated independently from the source code. This creates several issues:

- Manual synchronization is time-consuming and error-prone
- Team members unfamiliar with the current codebase struggle to determine if documentation and tests are up-to-date
- Inconsistencies emerge between documented API behavior and actual test coverage

In this project proposal the aim is to to automatically generate test cases from OpenAPI specifications to ensure tests stay synchronized with API documentation, and vice versa, throughout the development lifecycle.

2 Models

The project includes the use of four different metamodels.

- **OpenAPIMM** OpenAPI specifications
 - **TestCaseMM** Represent abstract test cases
 - **JUnitMM** JUnit test structure with REST Assured
 - **TestResultMM** Represent execution results.
1. **OpenAPIMM** – A metamodel representing OpenAPI specifications, including:
 - Endpoint paths (URLs and routes) with HTTP operations (GET, POST, PUT, DELETE, etc.)
 - Request/response schemas and data types of the specified data
 - Authentication and security schemes (OAuth2, API keys, JWT)
 - API metadata and documentation descriptions
 2. **TestCaseMM** – A metamodel representing abstract test cases in a platform-independent way, including:
 - Test suites and test case structures
 - Test preconditions and input data
 - Expected outputs and assertions based on predefined conditions
 - Test metadata
 3. **JUnittestMM** – A metamodel representing JUnit test structures with REST Assured integration, including:
 - Test classes and methods with annotations for API calls
 - HTTP request configurations (headers, parameters, body)
 - Response validation logic (status codes, response body, headers)
 - Setup and teardown operations for test execution
 4. **TestResultMM** – A metamodel representing test execution results:
 - Test outcomes (passed, failed, skipped, error)
 - Execution timestamps and duration information
 - Error messages and stack traces
 - Coverage metrics and test statistics

2.1 Tool integration

Gradle is a building automation tool and will consume the generated JUnit test code (Java files) and execute the tests. Gradle will produce test execution results in JUnit XML format. Those generated results will produce a TestResult model through a T2M transformation to capture test outcomes, execution times, and error messages.

The second tool is **Swagger Editor/OpenAPI tools**, which will produce or validate the OpenAPI specification files that serve as input to the system. Model compliance is ensured by validating against the OpenAPI specification standard.

The third integration involves the generated tests and execute them automatically, producing JUnit XML results that feed back into the system.

Model compliance:

The generated test code complies with:

- JUnit testing framework standards
- REST Assured API endpoint testing
- Gradle project structure conventions
- JUnit XML result format specification

2.2 How will the model be created, updated?

The **OpenAPI model** will be created in three ways:

- Manually created by developers using OpenAPI editors (Swagger Editor, VS Code with OpenAPI extensions)
- Generated from existing API implementations using documentation tools
- Imported from existing YAML/JSON specification files through a T2M transformation

The **TestCase model** is created automatically from an M2M transformation from the OpenAPI model. For each endpoint and HTTP method, test cases are generated including positive tests (valid requests), negative tests (invalid parameters, authentication failures), and boundary value tests based on schema constraints. The TestCase model gets updated automatically every time new test results come in. The update adds information about which tests passed or failed and execution time of the test and other relevant information.

The **JUnit model** is created automatically from an M2M transformation from the TestCase model, mapping abstract test structures to JUnit specific implementations with REST Assured.

The **TestResult model** is created automatically when gradle runs the tests and creates JUnit XML files. This happens through a T2M transformation.

3 Transformations

3.1 How will you combine transformations?

The tools needed for implementing this project, will be the tools covered in the course: Eclipse Modeling Tools with the Acceleo and QVTo plugins. The four metamodels (OpenAPIMM, TestCaseMM, JUnitMM, and TestResultMM) will be defined using Ecore within Eclipse EMF.

For the M2M transformations, I plan to use QVTo. This includes the transformations from OpenAPI to TestCase, TestCase to JUnit, and the feedback transformation from TestResult back to TestCase. QVTo is well suited for the M2M operations because it allows expressing complex mapping rules between metamodel elements.

Acceleo will be used for generating text outputs, . This covers both the Java test code generation and the Markdown documentation. Acceleos approach seems ideal for producing structured code files with proper formatting.

The T2M transformation that reads JUnit XML files will be implemented using Java's built-in XML parsing capabilities, creating TestResult model instances from the execution data.

3.2 Transformation Overview

The following table summarizes all transformations, their types, tools, and descriptions.

Transformation	Type	Tool	Description
OpenAPI → TestCase	M2M	QVTo	Derives abstract test cases from API operations.
TestCase → JUnit	M2M	QVTo	Maps abstract test logic to JUnit structures.
JUnit → Java	M2T	Acceleo	Generates executable Java test code.
JUnit XML → TestResult	T2M	Java parser	Imports test outcomes as model elements.
TestResult → TestCase	M2M	QVTo	Enriches the TestCase model with feedback.

Table 1: Summary of model transformations, types, tools, and purposes.

3.3 Transformation Flows

The system operates through two distinct pipelines:

Pipeline 1: Generating Tests

Three transformations execute in a chain: OpenAPI→TestCase using QVTo, then TestCase→JUnit also with QVTo, and finally JUnit→Java using Acceleo. Each step depends on the previous one completing, which is why they must run in order. The final output is working Java test code derived from the API specification.

Running separately, the documentation generation (OpenAPI→Markdown) can happen whenever needed since it only reads from the OpenAPI model. This transformation also uses Acceleo but doesn't depend on the other transformations.

Pipeline 2: Processing Results

When Gradle finishes running the tests, this pipeline activates. First, the XML result files are read and converted into a TestResult model (T2M using Java). Then, a QVTo transformation takes this execution data and enriches the TestCase model with it. This adds useful information like pass rates, execution times, and flags unreliable tests.

When the feedback reveals issues and several tests are consistently failing or there are obvious gaps in coverage Pipeline 1 can restart to produce updated tests.

Automation Setup

The system runs on its own through listeners built into EMF that monitor the OpenAPI model. When someone edits the API specification, the listener catches this change and kicks off Pipeline 1 automatically. It will create a Java orchestration class that manages calling these transformations in the right sequence. This automation means developers only need to maintain their API specifications, and the test code updates itself accordingly.

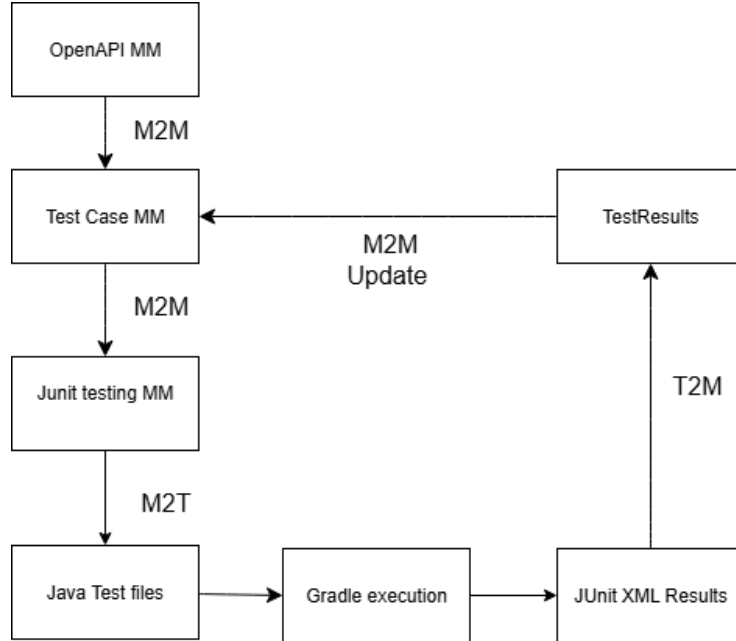


Figure 1: Figure displays the transformation flow and type of transformation

References

- [1] openapi2021 OpenAPI Initiative (2021). *OpenAPI Specification v3.0.3*. <https://spec.openapis.org/oas/v3.0.3>
- [2] Swagger (2023). *OpenAPI Specification Documentation*. <https://swagger.io/specification/>
- [3] JUnit Team (2023). *JUnit 5 User Guide*. <https://junit.org/junit5/docs/current/user-guide/>
- [4] Gradle Inc. (2023). *Gradle Build Tool*. <https://gradle.org/>